

Deccan Education Society's

**Kirti M. Doongursee College of Arts, Science and Commerce [Autonomous], Dadar
(West), Mumbai-400 028.**



M.Sc. [Computer Science]

25PSMJ91 Web3 Technologies

Seat Number []

Department of Computer Science and Information Technology

KIRTI M. DOONGURSEE COLLEGE OF ARTS, SCIENCE AND COMMERCE

(Affiliated to University of Mumbai)

DEPARTMENT OF COMPUTER SCIENCE

MASTER OF SCIENCE (COMPUTER SCIENCE)

SEMESTER III — ACADEMIC YEAR 2026-2027

SUBJECT: WEB3 TECHNOLOGIES

CERTIFICATE

This is to certify that **Ms. Dipali Ganesh Patil** of M.Sc. (Computer Science) with Seat No. _____ (Roll No. **21**) has completed the project “**BlockVerify**” of Paper – Web3 Technologies in this College during the year **2026-2027**.

Lecturer-In-Charge

Date: / /2026

Examined by:

Remarks:

Date:

INDEX – BLOCKVERIFY

Sr. No.	Contents	Page No.
1	Introduction	1
2	Problem Statement	3
3	Objectives	4
4	Literature Review / Existing System	5
5	Proposed System	7
6	System Architecture	9
7	Technologies Used	12
8	Blockchain Architecture	13
9	Smart Contract Design	17
10	Implementation	20
11	Source Code	22
12	Screenshots and Diagrams	32
13	Testing	38
14	Results and Discussion	41
15	Limitations	43
16	Future Scope	44
17	Conclusion	45

INTRODUCTION

Academic certificates are important documents because they show a student's educational qualifications. Colleges and universities issue certificates after students complete their courses or degrees. These certificates are often required for jobs, higher studies, internships, and other official purposes. Therefore, it is important that certificate information is correct, safe, and easy to verify.

In the traditional system, certificate records are usually stored by colleges or universities. When someone wants to verify a certificate, they may need to contact the institution or check its records. This process can take time, especially when the information is not easily available.

There is also a risk of fake certificates or changes to certificate records. In a centralized system, records are managed by the organization that controls the system. Blockchain technology provides another way to store records. It records information through transactions on a blockchain, making the stored data difficult to change without a new transaction.

BlockVerify is a blockchain-based academic certificate verification system developed as a Web3 application. It is based on the Blockchain-Based Certificate Verification System theme given in the project instructions. The main purpose of BlockVerify is to make certificate issuance and verification simple and transparent.

In this system, an authorized institution can issue a certificate by entering basic information such as Certificate ID, student name, and course name. The certificate information is stored using a Solidity smart contract on an Ethereum-compatible blockchain. Each certificate has a unique Certificate ID that can be used to find and verify its details.

The system provides a public verification page where users can enter a Certificate ID and check the certificate information stored on the blockchain. The page displays details such as the student's name, course, issuer address, issue date, and validity status. This helps users check certificate records without directly contacting the institution.

BlockVerify also includes an admin section for authorized institutions. The administrator can connect their wallet and issue new certificates, update existing certificate details, and revoke certificates when required. The system also supports certificate expiry management, allowing the administrator to set or update an expiry date.

An Institution Certificate Dashboard is included to help administrators manage their issued certificates. It displays summary information, such as total, valid, revoked, and expired certificates. The administrator can search certificates using the Certificate ID, student name, or course name, filter records by status, and view individual certificate details. The dashboard can also be refreshed to display updated records after certificate operations.

Only the authorized administrator wallet can perform restricted actions, such as issuing, updating, and revoking certificates. MetaMask is used to connect the wallet to the application, while Ethers.js helps the frontend communicate with the smart contract.

The project uses Solidity to develop the smart contract and Ganache as a local blockchain for development and testing. Remix IDE is used for smart contract development and deployment. The user interface is created using HTML, CSS, and JavaScript, and Visual Studio Code is used for frontend development.

The basic working of the system is:

Institution → Issue Certificate → Store on Blockchain → Generate Certificate ID → Verify Certificate

BlockVerify is developed as a working academic prototype to demonstrate the practical use of blockchain and smart contracts in certificate verification. It shows how Web3 technologies can be used to issue, verify, update, revoke, and manage academic certificate records through a simple web application.

The main aim of this project is to provide a simple and transparent certificate verification system while demonstrating the use of blockchain, smart contracts, MetaMask, Ethers.js, and decentralized application concepts.

PROBLEM STATEMENT

Academic certificates are important for proving a student's education and qualifications. However, checking whether a certificate is genuine can sometimes be a difficult and time-consuming process.

In a traditional system, certificate information is generally maintained by the institution that issued the certificate. When another person or organization wants to verify a certificate, they may need to contact the institution or check its records manually. This can take additional time and effort.

Another problem is that certificate information should not be changed without proper authorization. There is a need for a system where certificate records are stored in a controlled manner and where only the authorized institution can issue, update, or revoke certificates.

A simple and reliable way is also needed for anyone to verify a certificate. The user should be able to provide a unique Certificate ID and receive the certificate details and its current status without going through a complicated verification process.

The BlockVerify project addresses these problems by using blockchain technology and a Solidity smart contract. The system allows the authorized institution wallet to issue certificates and manage their records. The smart contract stores the Certificate ID, student name, course, issuer address, issue date, expiry date, and validity status.

The system also provides a public verification page where a user can enter a Certificate ID and check the certificate information. The smart contract restricts issue, update, and revoke operations to the authorized institution wallet. It also supports certificate expiry management and provides an institution dashboard to view, search, filter, and manage issued certificates.

Therefore, the main problem addressed by BlockVerify is the need for a simple blockchain-based system that can securely manage academic certificate records and allow easy verification of their current status.

3. OBJECTIVES

The main objective of the BlockVerify project is to develop a simple blockchain-based system for managing and verifying academic certificates.

The objectives of the project are:

1. To create a unique Certificate ID for every academic certificate.
2. To store important certificate details using a blockchain-based smart contract.
3. To allow the authorized institution to issue new academic certificates.
4. To provide a simple public system for verifying a certificate using its Certificate ID.
5. To display important certificate information such as student name, course, issuer, issue date, expiry date, and validity status.
6. To allow the authorized institution to update certificate details when required.
7. To provide a revoke feature for certificates that are no longer valid.
8. To support certificate expiry management by allowing the institution to set or update expiry dates.
9. To restrict certificate management operations to the authorized institution wallet.
10. To provide an institution dashboard for viewing and managing issued certificates.
11. To allow the institution to search certificates and filter them by status.
12. To display certificate statistics, including total, valid, revoked, and expired certificates.
13. To demonstrate the use of Solidity smart contracts and Ethereum-compatible blockchain technology.
14. To develop a working Web3 application using MetaMask, Ethers.js, HTML, CSS, and JavaScript.

4. LITERATURE / EXISTING SYSTEM

Academic certificate verification is generally handled through the records maintained by colleges and universities. When a student completes a course, the institution issues a certificate and keeps the related information in its records.

In the traditional system, a person or organization that wants to verify a certificate may need to contact the issuing institution or use the verification method provided by that institution. The verification process therefore depends on the institution having the required records available and responding to the verification request.

Certificate information is commonly managed through physical documents or centralized digital records. These methods are useful for storing certificate information, but the verification process can still require manual checking. If the records are maintained separately by different institutions, there is also no single common system for checking certificates.

Another important issue is access control. Only the authorized institution should be able to issue or change certificate records. A proper system therefore needs to control who can perform certificate management operations.

Blockchain provides a different approach for maintaining such records. Blockchain records transactions on a distributed ledger, and smart contracts can be used to define rules for how records are created and managed. This makes blockchain suitable for applications where the history and validity of records need to be checked.

The BlockVerify project uses this blockchain approach for academic certificate verification. The system stores certificate information through a Solidity smart contract and uses a unique Certificate ID to identify each certificate. It also supports certificate updates, revocation, and expiry management through authorized operations.

The existing system can be understood through the following basic process:

Institution → Certificate Issued → Institution Records → Verification Request →
Manual/Institution Verification

BlockVerify changes this process by allowing the authorized institution to record certificate information on the blockchain and allowing users to directly check the certificate using its Certificate ID.

The project also includes an institution dashboard where the administrator can view issued certificates, search records, filter certificates by status, view certificate details, and check summary statistics. These features help the institution manage its certificate records through one interface.

The project focuses on the main operations of issuing, verifying, updating, and revoking academic certificates, along with expiry management and dashboard-based record management. It is developed as a local academic prototype using blockchain and smart contract technology.

This existing-system analysis helped identify the need for a simple blockchain-based certificate verification application that is easy to use and demonstrates the practical use of smart contracts.

5. PROPOSED SYSTEM

The proposed system, BlockVerify, is a blockchain-based academic certificate verification application. The main purpose of the system is to make the process of issuing, managing, and verifying certificates simple and easy to use.

In BlockVerify, the authorized institution first connects its wallet to the application. The institution can then issue a certificate by entering a unique Certificate ID, student name, and course or degree name. The certificate information is stored through the smart contract on the blockchain.

Each certificate has a unique Certificate ID. A user can enter this ID on the public verification page to check the certificate. The system displays the certificate details along with the issue date, expiry date, and current validity status. The public verification process does not require the user to have access to the institution portal.

The institution can also manage certificates after they have been issued. The authorized institution wallet can update certificate details when a correction is required. It can also revoke a certificate when it is no longer valid and set or update its expiry date.

The smart contract controls the certificate management operations. Only the wallet stored as the contract owner can perform the issue, update, and revoke operations. This provides a simple access-control mechanism for the prototype.

The proposed system also includes an Institution Certificate Dashboard. It allows the authorized institution to view its issued certificates and summary statistics, including total, valid, revoked, and expired certificates. The administrator can search certificates using the Certificate ID, student name, or course name, filter records by status, and view individual certificate details. The dashboard can be refreshed to display updated information after certificate operations.

The proposed system contains two main parts:

1. Public Verification Module

This module allows a user to enter a Certificate ID and view its recorded information and validity status. It helps users check certificate records without contacting the institution directly.

2. Institution Portal

This module allows the authorized institution to connect its wallet and issue, update, revoke, and manage certificate expiry. It also provides the Institution Certificate Dashboard for viewing, searching, filtering, and managing issued certificates.

The frontend communicates with the deployed smart contract using Ethers.js. The application creates an Ethers.js contract instance using the deployed contract address and ABI. MetaMask is used to connect the institution's wallet and approve blockchain transactions.

The overall proposed process is:

Institution Wallet → Issue Certificate → Blockchain Record → Certificate ID → Public Verification

For certificate management, the process is:

Authorized Institution → Dashboard → Search / Filter / View → Update / Revoke / Manage Expiry → Updated Blockchain Record

The proposed system demonstrates how blockchain and smart contracts can be used to manage academic certificate records and provide a simple public verification process. It is implemented as a local academic prototype for development and testing.

6. SYSTEM ARCHITECTURE

BlockVerify follows a simple Web3 decentralized application (dApp) architecture. The user interacts with the frontend through a web browser, while certificate operations are handled by the Solidity smart contract deployed on the local blockchain.

The main components of the system are:

6.1 User Interface

The frontend is developed using HTML, CSS, and JavaScript. It provides two main interfaces:

- Public certificate verification page
- Institution certificate management portal

The public page allows users to search for and verify certificates using a unique Certificate ID. The institution portal provides certificate management functions, including issuing, updating, revoking, and managing certificate expiry. It also includes an Institution Certificate Dashboard to view and manage issued certificates.

6.2 MetaMask Wallet

MetaMask is used to connect the user's blockchain wallet to the application. For institution operations, the authorized institution wallet is required.

The application requests the user's blockchain account through MetaMask and obtains the connected wallet address. The connected wallet is used to authorize and sign transactions for restricted certificate management operations.

6.3 Ethers.js

Ethers.js acts as the connection between the frontend and the blockchain. It is used to create the provider, signer, and smart contract instance.

The application creates a contract object using the deployed contract address, ABI, and signer. Ethers.js allows the frontend to call smart contract functions, retrieve certificate information, and send transactions for certificate management.

6.4 CertificateVerification Smart Contract

The Solidity smart contract is the main part of the system. It manages certificate records and controls the operations that can be performed on them.

The contract handles:

- Certificate issuing

- Certificate verification
- Certificate updating
- Certificate revocation
- Certificate expiry management
- Retrieving certificates issued by an institution
- Providing certificate statistics
- Institution wallet authorization through the owner address

The smart contract stores certificate details, including the Certificate ID, student name, course, issuer address, issue date, expiry date, and validity status.

6.5 Ganache Blockchain

Ganache is used as the local Ethereum-compatible blockchain for developing and testing the prototype. The smart contract is deployed on Ganache, and certificate management transactions are sent to it through the frontend.

The blockchain stores the certificate records and transaction history created by the smart contract. Ganache provides a local environment for testing the application's functionality.

6.6 Institution Certificate Dashboard

The Institution Certificate Dashboard is part of the admin portal. It allows the authorized institution to view and manage its issued certificates.

The dashboard provides:

- Summary statistics for total, valid, revoked, and expired certificates
- Certificate search using Certificate ID, student name, or course name
- Status-based filtering
- A detailed view of individual certificates
- A refresh option to load updated certificate records

The dashboard retrieves certificate records associated with the connected institution wallet through the smart contract.

6.7 System Flow

The system has two main flows: public certificate verification and institution certificate management.

Public Verification Flow:

User → Public Verification Page → Enter Certificate ID → Ethers.js → Smart Contract → Certificate Details and Status

For public verification, the application uses a read-only blockchain connection to call the `verifyCertificate()` function and retrieve certificate information. The user does not need to connect an institution wallet to verify a certificate.

Institution Management Flow:

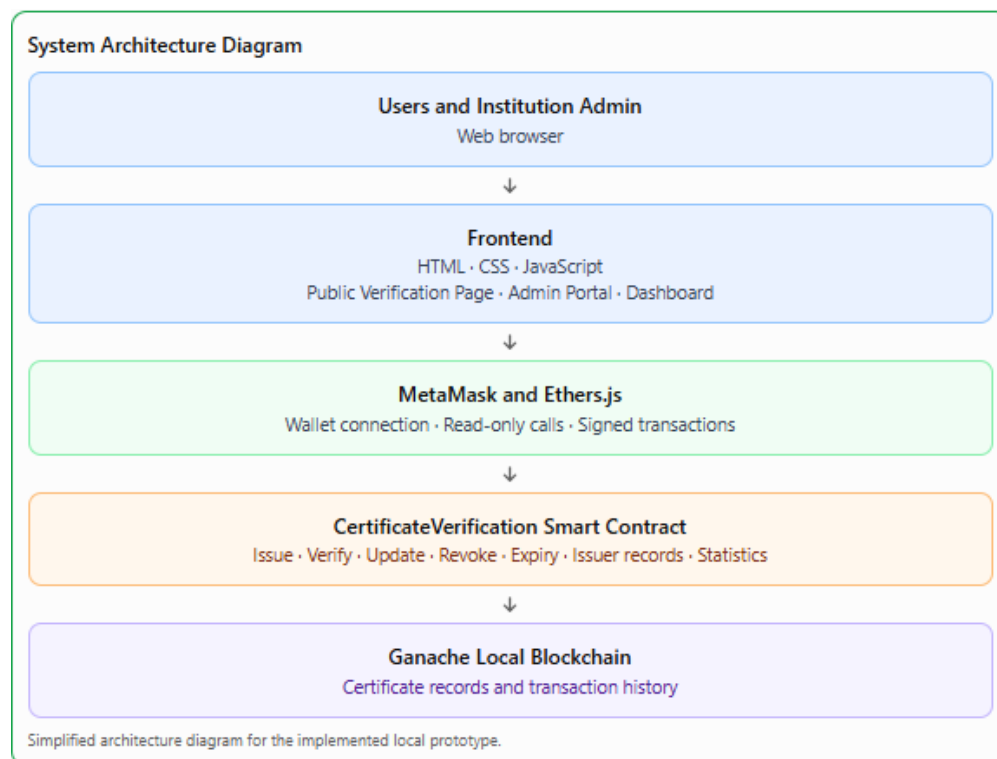
Institution Admin → MetaMask Wallet → Admin Portal → Wallet Authorization Check → Ethers.js → Smart Contract → Blockchain Transaction

For institution operations, the connected wallet acts as the signer and sends transactions to the smart contract. Before issuing, updating, or revoking a certificate, the application checks whether the connected wallet matches the contract owner.

Dashboard Flow:

Authorized Institution Wallet → Institution Dashboard → Retrieve Certificates → Search / Filter / View Details → Display Records and Statistics

The dashboard retrieves certificates associated with the connected institution wallet and displays the available records and summary statistics. Certificate updates, revocations, and expiry changes are handled through the authorized smart contract functions.



7. TECHNOLOGIES USED

BlockVerify is developed using different Web3 and web development technologies. Each technology has a specific role in making the certificate verification system work.

Technology	Purpose
Solidity 0.8.19	Used to create the CertificateVerification smart contract.
Ethereum-Compatible Blockchain	Used to store certificate records through the smart contract.
Ganache	Used as the local blockchain network for development and testing.
MetaMask	Used to connect the institution wallet with the blockchain and confirm transactions.
Ethers.js 6.13.5	Used to connect the frontend application with the smart contract.
HTML	Used to create the structure of the public verification and institution portal pages.
CSS	Used to design the user interface, buttons, forms, cards, header, hero section, and footer.
JavaScript	Used to handle user actions and interact with the smart contract.
Remix IDE	Used to write, compile, and deploy the Solidity smart contract.
Visual Studio Code	Used to create and manage the frontend project files.

8. BLOCKCHAIN ARCHITECTURE

BlockVerify uses an Ethereum-compatible blockchain architecture in which the main certificate management logic is handled by a Solidity smart contract. The smart contract is the central part of the system because it stores certificate records and controls who can issue, update, and revoke them. It also supports certificate expiry management and provides functions to retrieve certificate records and statistics.

The project uses Ganache as a local blockchain network for development and testing. MetaMask is used as the blockchain wallet, and Ethers.js is used to connect the web application with the deployed smart contract.

8.1 Main Components

1. Institution Wallet

The institution uses a blockchain wallet through MetaMask. The wallet address that deploys the smart contract becomes the contract owner. This address is treated as the authorized institution wallet and is required for restricted certificate management operations.

2. MetaMask

MetaMask connects the browser with the blockchain network. It is used to request the connected account and confirm transactions made by the institution. The application checks the connected wallet against the contract owner before allowing administrative operations.

3. Smart Contract

The CertificateVerification smart contract contains the rules for certificate management. It stores certificate details and provides functions for issuing, verifying, updating, and revoking certificates. It also supports expiry management, issuer-based certificate retrieval, and certificate statistics.

4. Ganache

Ganache provides the local blockchain environment used for the prototype. It allows transactions and smart contract operations to be tested locally without using a public blockchain network.

5. Frontend Application

The frontend provides the public verification page and the institution portal. JavaScript communicates with the smart contract using Ethers.js. The institution portal also includes a dashboard for viewing, searching, filtering, and managing issued certificates.

8.2 Certificate Data on Blockchain

Each certificate is stored with the following information:

- Certificate ID
- Student name
- Course
- Issuer wallet address
- Issue date
- Expiry date
- Validity status
- Existence status

The smart contract uses a mapping to connect a Certificate ID with its certificate record.

```
</> solidity
mapping(string => Certificate) private certificates;
```

This allows the certificate to be retrieved using its unique Certificate ID. The contract also maintains issuer-based certificate records to help the institution retrieve certificates associated with its wallet.

8.3 Blockchain Transaction Flow

For an institution operation, the process is:

Institution Wallet → MetaMask → Frontend Application → Ethers.js → Smart Contract → Ganache Blockchain

The institution connects its wallet through MetaMask and performs an administrative operation, such as issuing, updating, or revoking a certificate. The application checks the wallet authorization and uses Ethers.js to send the signed transaction to the smart contract. Ganache processes the transaction on the local blockchain.

For certificate verification, the application uses a read-only blockchain connection. It calls the `verifyCertificate()` function and retrieves the stored certificate information without sending a transaction.

For issuing, updating, and revoking certificates, the connected institution wallet signs and submits a blockchain transaction. The smart contract checks the authorization and applies the requested operation.

8.4 Overall Architecture

The complete Blockchain Architecture of BlockVerify can be represented as:

BlockVerify Blockchain Architecture

Users and Institution Admin

Public users verify certificates; institution admin manages records

Frontend Application

Public verification page · Admin portal · Institution dashboard

MetaMask and Ethers.js

Wallet connection · Authorization check · Read calls and signed transactions

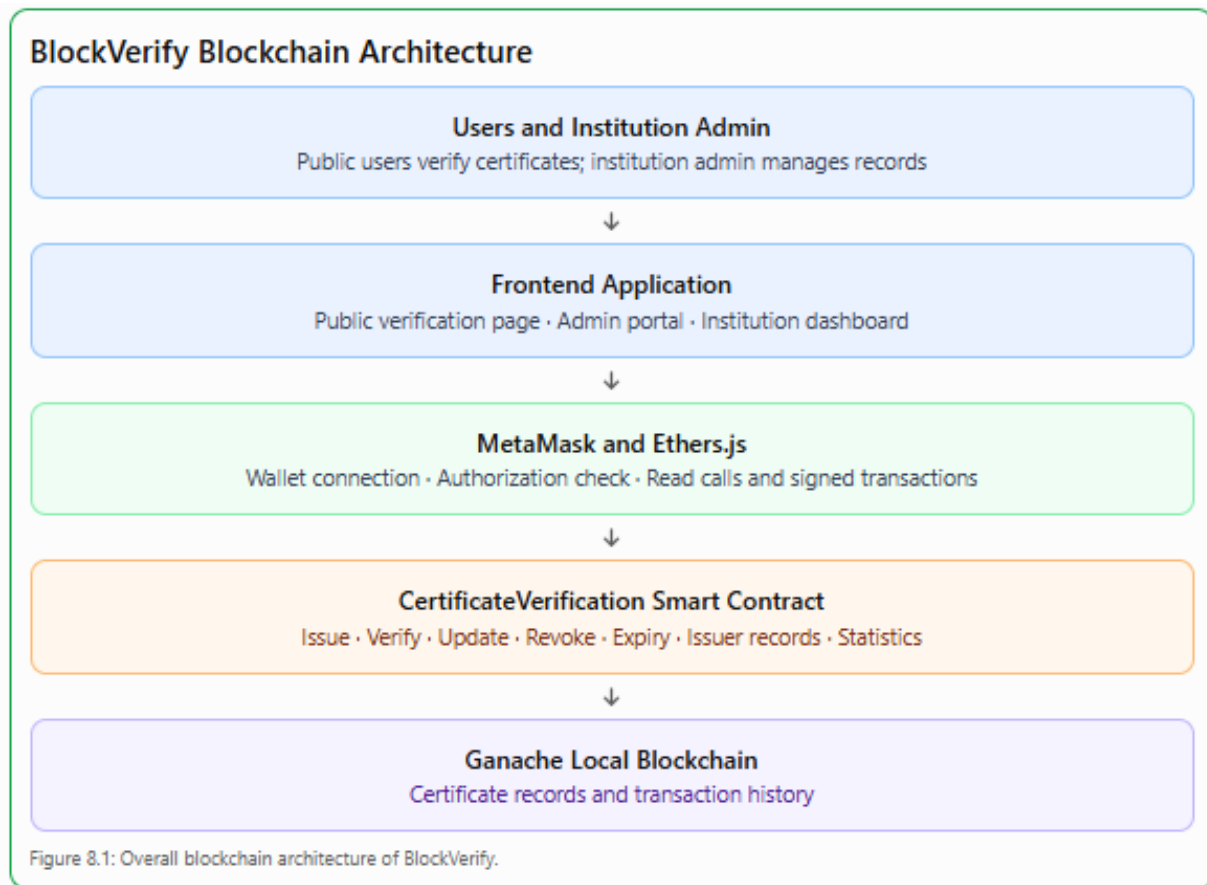
CertificateVerification Smart Contract

Issue · Verify · Update · Revoke · Expiry · Issuer records · Statistics

Ganache Local Blockchain

Certificate records and transaction history

Figure 8.1: Overall blockchain architecture of BlockVerify.



This architecture provides a simple connection between the user interface and the blockchain. Public users can read and verify certificate information, while the authorized institution can perform certificate management operations through the smart contract. The local Ganache network provides the environment for deploying and testing the application.

9. SMART CONTRACT DESIGN

The BlockVerify smart contract is written in Solidity 0.8.19. It manages academic certificate records and controls the main certificate operations, such as issuing, verifying, updating, revoking, and managing certificate expiry.

9.1 Certificate Structure

The contract stores the following information for each certificate:

- Certificate ID
- Student Name
- Course
- Issuer Address
- Issue Date
- Expiry Date
- Validity Status
- Existence Status

Each certificate is identified by a unique Certificate ID. The contract uses a mapping to store and retrieve certificate records.

9.2 Access Control

The wallet that deploys the contract becomes the owner. Only this authorized institution wallet can issue, update, or revoke certificates and perform other restricted management operations.

Public users can verify certificates and view their recorded information without having permission to change the records.

9.3 Main Functions

Function	Purpose
issueCertificate()	Creates a new certificate and stores its details.
verifyCertificate()	Retrieves certificate information and its validity status.
updateCertificate()	Updates the student's name and course details.
revokeCertificate()	Marks a certificate as revoked.
getCertificate()	Retrieves the complete stored certificate record.
certificateExists()	Checks whether a certificate exists.
getCertificateStatus()	Returns the certificate's validity status.
getCertificateIssuer()	Retrieves the issuer's wallet address.
getCertificateIssueDate()	Retrieves the certificate's issue date.
getTotalCertificates()	Returns the total number of certificates issued.
getCertificatesByIssuer()	Retrieves certificates associated with an issuer wallet.
getCertificateCountByStatus()	Returns certificate counts based on their status.
setCertificateExpiry()	Sets or updates a certificate's expiry date.
isCertificateExpired()	Checks whether a certificate has expired.

The contract uses a mapping to store certificates by their unique Certificate ID:

```
mapping(string => Certificate) private certificates;
```

This mapping connects each Certificate ID with its corresponding certificate record and allows the contract to retrieve certificate information.

9.4 Events

The contract uses events to record important certificate-related actions on the blockchain:

- CertificateIssued – Records when a new certificate is issued.
- CertificateUpdated – Records when certificate details are updated.
- CertificateRevoked – Records when a certificate is revoked.

These events help track important certificate operations and provide a record of actions performed through the smart contract.

9.5 Certificate Expiry Management

The smart contract includes expiry management functions. The authorized institution can set or update a certificate's expiry date using `setCertificateExpiry()`. The `isCertificateExpired()` function checks whether the certificate has passed its expiry date.

This feature helps distinguish certificates that are currently valid from those that have expired. Expiry is checked separately from the certificate's stored validity status.

9.6 Issuer-Based Certificate Retrieval and Statistics

The contract provides functions to retrieve certificates associated with an issuer wallet and to obtain certificate statistics.

The `getCertificatesByIssuer()` function retrieves certificates issued by a particular institution wallet. The `getCertificateCountByStatus()` function provides counts based on certificate status, while `getTotalCertificates()` returns the total number of certificates.

These functions support the Institution Certificate Dashboard, where the authorized institution can view its certificate records and summary statistics.

Overall, the smart contract provides the main functionality required for issuing, verifying, updating, revoking, and managing academic certificates. It also supports expiry checks, issuer-based retrieval, and statistics for the BlockVerify application.

10. IMPLEMENTATION

The BlockVerify system is implemented as a simple Web3 application that connects the frontend with the CertificateVerification smart contract. It provides public certificate verification and an Institution Portal for managing certificates.

10.1 Public Verification Module

The public page allows a user to enter a Certificate ID and verify it. The application calls the smart contract's `verifyCertificate()` function and displays the certificate details, issue date, issuer address, expiry date, and current status. Users can check whether a certificate is valid, revoked, or expired without connecting a wallet.

10.2 Institution Portal

The Institution Portal is used by the authorized institution to manage certificates. The institution connects its MetaMask wallet before performing any operation.

The application checks whether the connected wallet matches the contract owner. Only the authorized wallet can issue, update, or revoke certificates and manage their expiry dates.

10.3 Certificate Issuing

To issue a certificate, the institution enters the Certificate ID, student name, and course. The application sends the information to the `issueCertificate()` smart contract function. After the transaction is confirmed, the certificate is stored on the blockchain with its issuer address and issue date.

10.4 Certificate Updating

The institution can update the student name and course of an existing valid certificate. The application first checks that the certificate exists and has not been revoked, and then sends the update transaction to the smart contract.

10.5 Certificate Revocation

The institution can revoke an existing certificate by entering its Certificate ID. The smart contract changes its validity status to false, and the application displays the certificate as revoked during verification.

10.6 Certificate Expiry Management

The institution can set or update the expiry date of a certificate using the `setCertificateExpiry()` function. The application uses the smart contract's `isCertificateExpired()` function to check whether the certificate has passed its expiry date. The expiry status is also considered during public verification and in the Institution Certificate Dashboard.

10.7 Institution Certificate Dashboard

The Institution Certificate Dashboard allows the connected institution to view and manage certificates issued by its wallet. It retrieves the institution's certificates using the `getCertificatesByIssuer()` function.

The dashboard displays summary statistics for total, valid, revoked, and expired certificates. It also provides a search option to find certificates by Certificate ID, student name, or course, and a status filter to view certificates according to their current status. The institution can select View Details to inspect a certificate's information and refresh the dashboard after certificate operations.

10.8 Frontend–Blockchain Connection

Ethers.js is used to connect the frontend with MetaMask and the deployed smart contract. The application creates a provider, signer, and contract instance using the deployed contract address and ABI.

MetaMask is used to connect the institution's wallet and approve blockchain transactions. Ethers.js communicates with the smart contract to perform operations such as issuing, verifying, updating, revoking, and managing certificate expiry. The frontend also retrieves certificate details and statistics for display.

Overall, the implementation connects the Public Verification Module, Institution Portal, and Institution Certificate Dashboard with the blockchain through Ethers.js and MetaMask. The smart contract manages certificate records, while the frontend provides an interface for public verification and authorized institution operations.

11. SOURCE CODE

This section contains the source code used to develop the BlockVerify prototype. The project uses Solidity for the smart contract and HTML, CSS, and JavaScript for the frontend.

11.1 Solidity Smart Contract

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.19;
```

```
contract CertificateVerification {
```

```
    address public owner;
```

```
    struct Certificate {
```

```
        string certificateId;
```

```
        string studentName;
```

```
        string course;
```

```
        address issuer;
```

```
        uint256 issueDate;
```

```
        uint256 expiryDate;
```

```
        bool isValid;
```

```
        bool exists;
```

```
    }
```

```
    mapping(string => Certificate) private certificates;
```

```
    mapping(address => string[]) private certificatesByIssuer;
```

```
    uint256 public totalCertificates;
```

```
    uint256 public validCertificates;
```

```

uint256 public revokedCertificates;

event CertificateIssued(
    string certificateId,
    string studentName,
    string course,
    address issuer,
    uint256 issueDate
);

event CertificateUpdated(
    string certificateId,
    string studentName,
    string course
);

event CertificateRevoked(string certificateId);
event CertificateExpirySet(string certificateId, uint256 expiryDate);

modifier onlyOwner() {
    require(
        msg.sender == owner,
        "Only authorized institution can perform this action"
    );
    _;
}

constructor() {
    owner = msg.sender;

```

```
}
```

```
// 1. Issue a new certificate
```

```
function issueCertificate(
```

```
    string memory _certificateId,
```

```
    string memory _studentName,
```

```
    string memory _course
```

```
) public onlyOwner {
```

```
    require(bytes(_certificateId).length > 0, "Certificate ID required");
```

```
    require(bytes(_studentName).length > 0, "Student name required");
```

```
    require(bytes(_course).length > 0, "Course required");
```

```
    require(!certificates[_certificateId].exists, "Certificate already exists");
```

```
    certificates[_certificateId] = Certificate({
```

```
        certificateId: _certificateId,
```

```
        studentName: _studentName,
```

```
        course: _course,
```

```
        issuer: msg.sender,
```

```
        issueDate: block.timestamp,
```

```
        expiryDate: 0,
```

```
        isValid: true,
```

```
        exists: true
```

```
    });
```

```
    certificatesByIssuer[msg.sender].push(_certificateId);
```

```
    totalCertificates++;
```

```
    validCertificates++;
```

```

emit CertificateIssued(
    _certificateId,
    _studentName,
    _course,
    msg.sender,
    block.timestamp
);
}

```

// 2. Verify certificate details and validity

```

function verifyCertificate(
    string memory _certificateId
) public view returns (
    string memory certificateId,
    string memory studentName,
    string memory course,
    address issuer,
    uint256 issueDate,
    bool isValid
) {
    require(certificates[_certificateId].exists, "Certificate not found");

    Certificate memory cert = certificates[_certificateId];
    bool valid = cert.isValid &&
        (cert.expiryDate == 0 || block.timestamp <= cert.expiryDate);
}

```

```

return (
    cert.certificateId,
    cert.studentName,
    cert.course,
    cert.issuer,
    cert.issueDate,
    valid
);
}

// 3. Update certificate details
function updateCertificate(
    string memory _certificateId,
    string memory _studentName,
    string memory _course
) public onlyOwner {
    require(certificates[_certificateId].exists, "Certificate not found");
    require(certificates[_certificateId].isValid, "Certificate revoked");
    require(
        certificates[_certificateId].expiryDate == 0 ||
        block.timestamp <= certificates[_certificateId].expiryDate,
        "Certificate expired"
    );
    require(bytes(_studentName).length > 0, "Student name required");
    require(bytes(_course).length > 0, "Course required");

    certificates[_certificateId].studentName = _studentName;

```

```

    certificates[_certificateId].course = _course;

    emit CertificateUpdated(_certificateId, _studentName, _course);
}

// 4. Revoke a certificate
function revokeCertificate(
    string memory _certificateId
) public onlyOwner {
    require(certificates[_certificateId].exists, "Certificate not found");
    require(certificates[_certificateId].isValid, "Certificate already revoked");

    certificates[_certificateId].isValid = false;
    validCertificates--;
    revokedCertificates++;

    emit CertificateRevoked(_certificateId);
}

// 5. Get complete certificate record
function getCertificate(
    string memory _certificateId
) public view returns (
    string memory certificateId,
    string memory studentName,
    string memory course,
    address issuer,

```

```

uint256 issueDate,
uint256 expiryDate,
bool isValid,
bool exists
) {
    Certificate memory cert = certificates[_certificateId];

    bool valid = cert.isValid &&
        (cert.expiryDate == 0 || block.timestamp <= cert.expiryDate);

    return (
        cert.certificateId,
        cert.studentName,
        cert.course,
        cert.issuer,
        cert.issueDate,
        cert.expiryDate,
        valid,
        cert.exists
    );
}

```

// 6. Check whether certificate exists

```

function certificateExists(
    string memory _certificateId
) public view returns (bool) {
    return certificates[_certificateId].exists;
}

```

```
}
```

```
// 7. Get certificate status
```

```
function getCertificateStatus(  
    string memory _certificateId  
) public view returns (bool) {  
    require(certificates[_certificateId].exists, "Certificate not found");  
  
    Certificate memory cert = certificates[_certificateId];  
    return cert.isValid &&  
        (cert.expiryDate == 0 || block.timestamp <= cert.expiryDate);  
}
```

```
// 8. Get certificate issuer
```

```
function getCertificateIssuer(  
    string memory _certificateId  
) public view returns (address) {  
    require(certificates[_certificateId].exists, "Certificate not found");  
    return certificates[_certificateId].issuer;  
}
```

```
// 9. Get certificate issue date
```

```
function getCertificateIssueDate(  
    string memory _certificateId  
) public view returns (uint256) {  
    require(certificates[_certificateId].exists, "Certificate not found");  
    return certificates[_certificateId].issueDate;
```

```
}
```

```
// 10. Get total number of certificates
```

```
function getTotalCertificates() public view returns (uint256) {  
    return totalCertificates;  
}
```

```
// 11. Get certificate IDs issued by a wallet
```

```
function getCertificatesByIssuer(  
    address _issuer  
) public view returns (string[] memory) {  
    return certificatesByIssuer[_issuer];  
}
```

```
// 12. Get valid and revoked certificate counts
```

```
function getCertificateCountByStatus()  
    public  
    view  
    returns (uint256 validCount, uint256 revokedCount)  
{  
    return (validCertificates, revokedCertificates);  
}
```

```
// 13. Set or update certificate expiry date
```

```
// Use 0 for no expiry; otherwise provide a future Unix timestamp.
```

```
function setCertificateExpiry(  
    string memory _certificateId,
```

```

    uint256 _expiryDate
) public onlyOwner {
    require(certificates[_certificateId].exists, "Certificate not found");
    require(certificates[_certificateId].isValid, "Certificate revoked");
    require(
        _expiryDate == 0 || _expiryDate > block.timestamp,
        "Expiry must be in the future"
    );

    certificates[_certificateId].expiryDate = _expiryDate;
    emit CertificateExpirySet(_certificateId, _expiryDate);
}

// 14. Check whether certificate has expired
function isCertificateExpired(
    string memory _certificateId
) public view returns (bool) {
    require(certificates[_certificateId].exists, "Certificate not found");

    uint256 expiry = certificates[_certificateId].expiryDate;
    return expiry != 0 && block.timestamp > expiry;
}
}

```

12. SCREENSHOTS / DIAGRAMS

This section contains the screenshots and diagrams of the BlockVerify system. The screenshots show the main pages and important operations performed during testing.

12.1 System Architecture Diagram

The system architecture diagram shows the connection between the BlockVerify frontend, MetaMask, Ethers.js, smart contract, and Ganache blockchain.

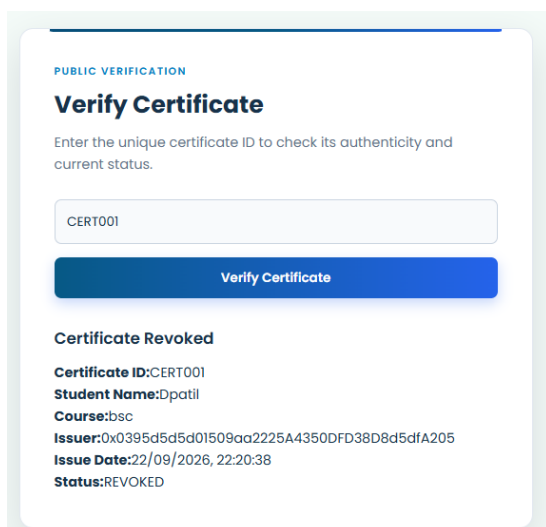
Figure 12.1: BlockVerify System Architecture



12.2 Public Verification Page

This screenshot shows the main BlockVerify public page. The user can enter a unique Certificate ID and verify the certificate without using the institution portal.

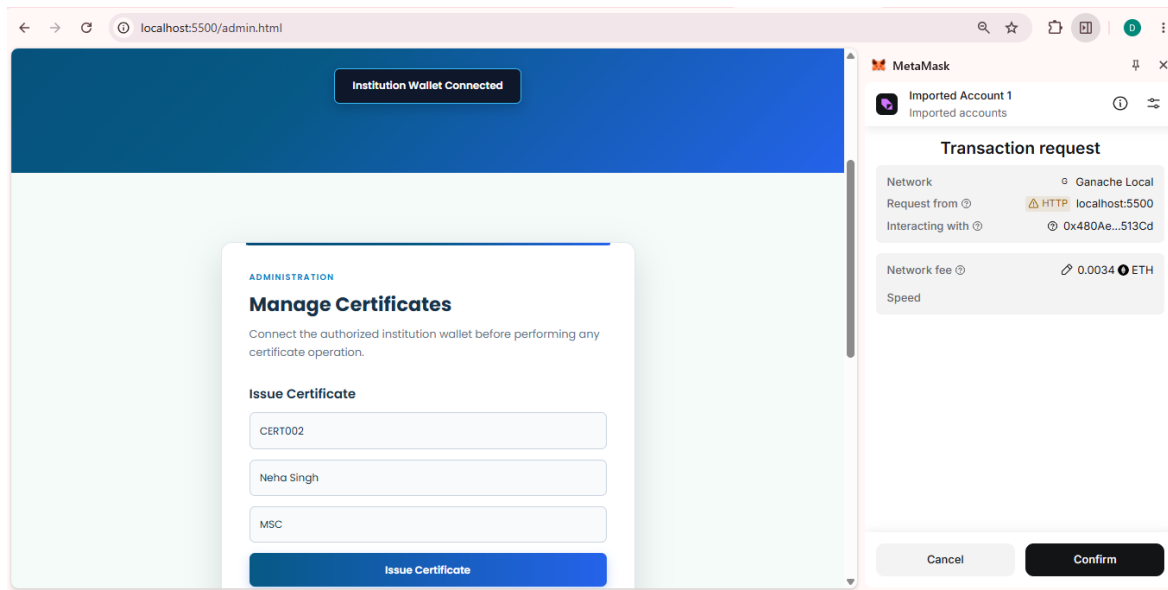
Figure 12.2: BlockVerify Public Verification Page



12.3 Institution Portal

This screenshot shows the Institution Portal where the authorized institution can connect its wallet and manage certificates.

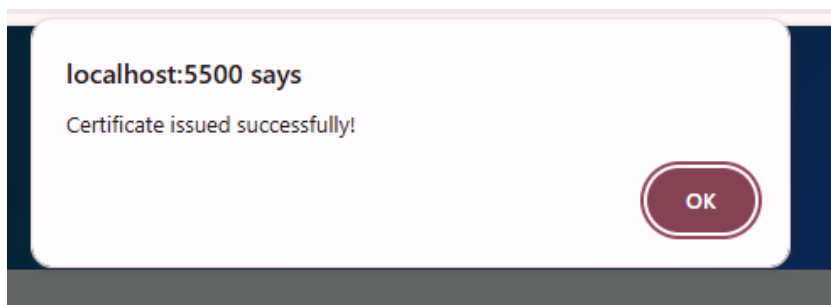
Figure 12.3: Institution Certificate Management Portal



12.4 Certificate Issued Successfully

This screenshot shows the successful issuing of an academic certificate. The institution enters the Certificate ID, student name, and course, and confirms the transaction through MetaMask.

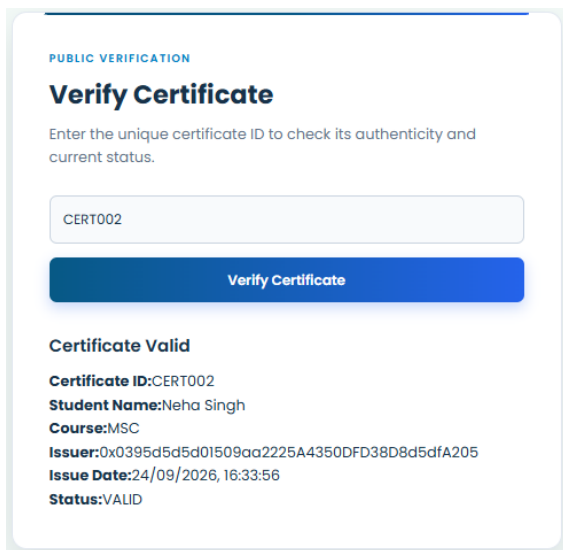
Figure 12.4: Certificate Issued Successfully



12.5 Valid Certificate Verification

This screenshot shows the result after entering a valid Certificate ID. The system displays the certificate details, issuer address, issue date, and **VALID** status.

Figure 12.5: Valid Certificate Verification Result



PUBLIC VERIFICATION

Verify Certificate

Enter the unique certificate ID to check its authenticity and current status.

CERT002

Verify Certificate

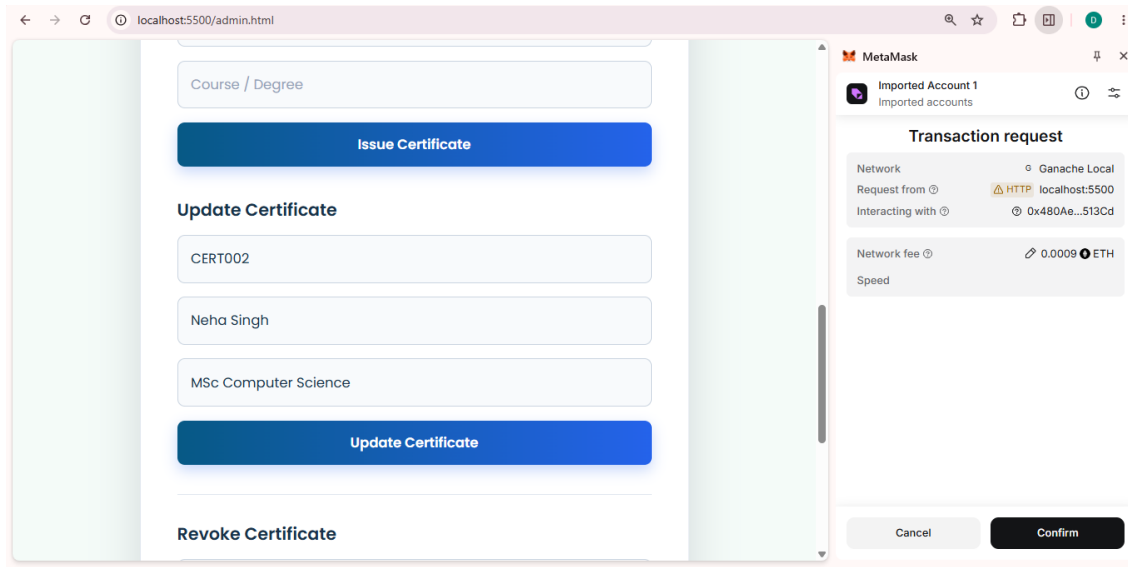
Certificate Valid

Certificate ID: CERT002
Student Name: Neha Singh
Course: MSC
Issuer: 0x0395d5d01509aa2225A4350DFD38D8d5dfA205
Issue Date: 24/09/2026, 16:33:56
Status: VALID

12.6 Certificate Updated

This screenshot shows the successful update of certificate information by the authorized institution.

Figure 12.6: Certificate Updated Successfully



Course / Degree

Issue Certificate

Update Certificate

CERT002

Neha Singh

MSc Computer Science

Update Certificate

Revoke Certificate

MetaMask

Imported Account 1
Imported accounts

Transaction request

Network: Ganache Local

Request from: localhost:5500

Interacting with: 0x480Ae...513Cd

Network fee: 0.0009 ETH

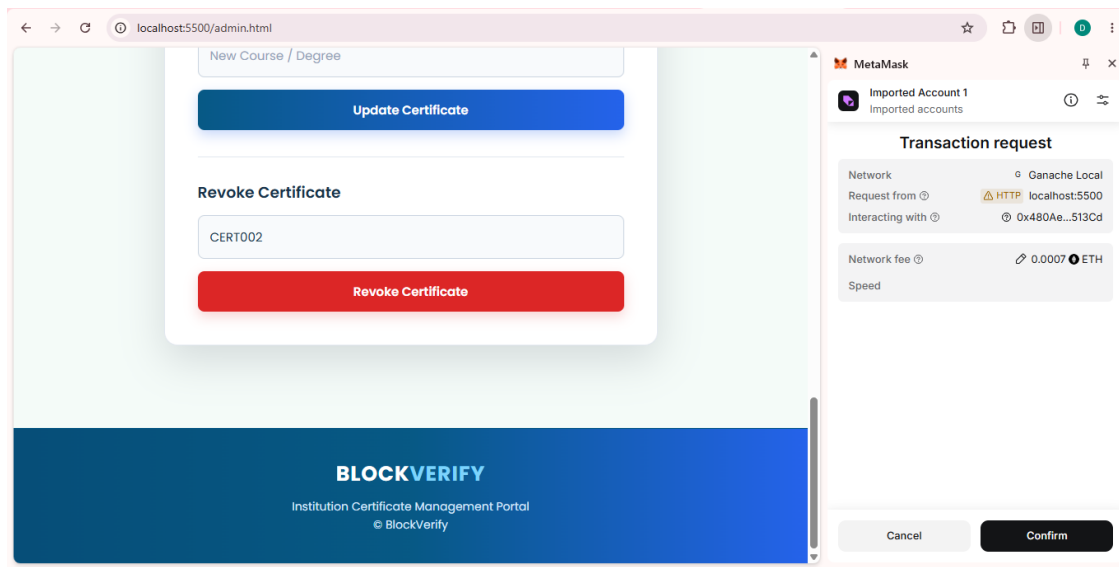
Speed

Cancel Confirm

12.7 Certificate Revoked

This screenshot shows a certificate after it has been revoked by the authorized institution. During verification, the system displays the certificate as **REVOKED**.

Figure 12.7: Revoked Certificate Verification Result



12.8 Invalid Certificate ID

This screenshot shows the message displayed when a user enters a Certificate ID that does not exist in the blockchain records.

Figure 12.8: Certificate Not Found

PUBLIC VERIFICATION

Verify Certificate

Enter the unique certificate ID to check its authenticity and current status.

Verify Certificate

Certificate Not Found

No certificate exists with this Certificate ID.

12.9 Certificate Expiry Management

This screenshot shows the expiry date management feature in the Institution Portal. The authorized institution can set or update the expiry date of an existing certificate.

Figure 12.9: Certificate Expiry Date Management

Manage Certificate Expiry

Set an expiry date for an existing certificate.

BV001

Expiry Date

09/29/2026



Set / Update Expiry

12.10 Institution Certificate Dashboard

This screenshot shows the Institution Certificate Dashboard, where the authorized institution can view certificates issued by its connected wallet. It displays summary statistics, including total, valid, revoked, and expired certificates.

Figure 12.10: Institution Certificate Dashboard

My Issued Certificates

View and manage certificates issued by the connected institution wallet.

Total Issued

3

Valid

2

Revoked

1

Expired

0

Search by ID, student name or

All Statuses



Refresh

Certificate ID	Student Name	Course / Degree	Status
BV2026001	Dipali Patil	MSc Computer Science - Updated	VALID
BV002	Rahul Sharma	M.com	REVOKE D

12.11 Certificate Details View

This screenshot shows the detailed information of a selected certificate in the Institution Certificate Dashboard. The institution can view the certificate information and its current status.

Figure 12.11: Certificate Details View

The interface shows a search bar with the text 'neha', a dropdown menu set to 'All Statuses', and a 'Refresh' button. Below this is a table with the following data:

Student Name	Course / Degree	Status	Action
Neha	Hsc	VALID	View Details

Below the table, it says 'Showing 1 of 3 certificates' with a scrollbar.

Certificate Details

Certificate ID: BV001

Student Name: Neha

Course / Degree: Hsc

Issuer Wallet:
0x0395d5d5d01509aa2225A4350DFD38D8d5dfA205

Issue Date: 27/09/2026, 23:31:31

Expiry Date: 29/09/2026, 23:59:59

Status: VALID

[Close Details](#)

12.12 Certificate Search and Status Filter

This screenshot shows the search and status filter features of the Institution Certificate Dashboard. The institution can search certificates using the Certificate ID, student name, or course, and filter records by their status.

Figure 12.12: Certificate Search and Status Filter

The interface shows a search bar with the text 'Di', a dropdown menu set to 'All Statuses', and a 'Refresh' button. Below this is a table with the following data:

Certificate ID	Student Name	Course / Degree	Status
BV2026001	Dipali Patil	MSc Computer Science - Updated	VALID

Below the table, it says 'Showing 1 of 3 certificates' with a scrollbar.

13. TESTING

Testing was performed to check whether the BlockVerify system works correctly and whether certificate operations are properly restricted to the authorized institution wallet.

The project was tested using Ganache, MetaMask, and the deployed CertificateVerification smart contract. Different test cases were performed for certificate issuing, verification, updating, revocation, expiry management, dashboard operations, and invalid or unauthorized operations.

13.1 Testing Objectives

The main objectives of testing were:

- To check whether the institution can connect its authorized wallet and issue certificates.
- To check whether users can verify certificates using the Certificate ID.
- To check whether certificate details can be updated correctly.
- To check whether certificates can be revoked.
- To check whether unauthorized wallets are prevented from performing restricted operations.
- To check the system response for invalid Certificate IDs.
- To verify that a revoked certificate is shown as revoked.
- To check whether the institution can set or update certificate expiry dates.
- To verify whether expired certificates are identified correctly.
- To check whether the Institution Certificate Dashboard displays the institution's certificates and statistics.
- To verify that certificate search, status filtering, details view, and dashboard refresh work correctly.

13.2 Test Cases

Test Case	Test Performed	Expected Result	Result
TC01	Connect authorized institution wallet	Institution wallet should connect successfully	Pass
TC02	Issue new certificate	Certificate should be stored successfully	Pass
TC03	Verify valid certificate	Correct certificate details should be displayed	Pass
TC04	Issue duplicate Certificate ID	System should reject the duplicate	Pass
TC05	Update certificate	Student/course details should be updated	Pass
TC06	Verify updated certificate	Updated information should be displayed	Pass
TC07	Revoke certificate	Certificate should become revoked	Pass
TC08	Verify revoked certificate	Status should show REVOKED	Pass
TC09	Verify invalid Certificate ID	Certificate Not Found should be displayed	Pass
TC10	Unauthorized wallet attempts issue	Operation should be rejected	Pass
TC11	Unauthorized wallet attempts update	Operation should be rejected	Pass
TC12	Unauthorized wallet attempts revoke	Operation should be rejected	Pass
TC13	Set or update certificate expiry date	Expiry date should be saved successfully	To be tested
TC14	Verify an expired certificate	System should identify and display the expired status	To be tested
TC15	Open Institution Certificate Dashboard	Dashboard should display certificates issued by the connected institution wallet	To be tested
TC16	Check dashboard statistics	Total, valid, revoked, and expired certificate counts should be displayed	To be tested

TC17	Search certificate in dashboard	Matching certificate should be displayed using Certificate ID, student name, or course	To be tested
TC18	Apply status filter	Certificates matching the selected status should be displayed	To be tested
TC19	View certificate details	Selected certificate's details should be displayed	To be tested
TC20	Refresh dashboard after a certificate operation	Dashboard should retrieve and display the updated certificate records	To be tested

14. RESULTS

The BlockVerify prototype was successfully developed and tested as a blockchain-based academic certificate verification system.

The main results of the project are:

- The authorized institution can issue a new certificate using a unique Certificate ID.
- Certificate details such as student name and course are stored through the smart contract.
- Users can verify a certificate by entering its Certificate ID.
- The system displays the student name, course, issuer address, issue date, expiry date, and current status.
- The authorized institution can update certificate details when required.
- The institution can revoke a certificate and change its status to REVOKED.
- Duplicate Certificate IDs are not accepted.
- Invalid Certificate IDs show a certificate-not-found message.
- Unauthorized wallets cannot perform issue, update, or revoke operations.
- The institution can set or update certificate expiry dates through the smart contract.
- The system can identify certificates that have expired.
- The Institution Certificate Dashboard allows the authorized institution to view certificates issued by its connected wallet.
- The dashboard provides summary statistics for total, valid, revoked, and expired certificates.
- The institution can search certificates by Certificate ID, student name, or course and filter them by status.
- The institution can view certificate details and refresh the dashboard to retrieve updated records.
- MetaMask and Ethers.js successfully connect the frontend with the deployed smart contract.

The testing showed that the main functions of BlockVerify worked as expected. The system demonstrated the use of blockchain and smart contracts for storing and verifying academic certificate records. The additional expiry management and dashboard features support certificate tracking and administration.

Overall, the project provides a working prototype for issuing, verifying, updating, revoking, and managing academic certificates using blockchain technology. It demonstrates how an Ethereum-compatible blockchain, Solidity smart contract, MetaMask, and Ethers.js can be used together to develop a certificate verification application

15. LIMITATIONS

Although BlockVerify is a working prototype, it has some limitations:

- 1. Local Blockchain** – The project uses Ganache for development and testing instead of a public blockchain network.
- 2. Manual Data Entry** – Certificate information is entered manually by the authorized institution.
- 3. Wallet-Based Authorization** – The system identifies the authorized institution using its blockchain wallet address. It does not verify the real-world identity or accreditation of the institution.
- 4. Prototype-Level Security** – The project is developed for academic demonstration and would require additional security measures, audits, and testing before real-world deployment.
- 5. No Institutional Database Integration** – The prototype does not connect with any real college or university database.
- 6. Wallet Dependency** – The institution needs MetaMask and the correct authorized wallet to perform certificate management operations.
- 7. Limited Certificate Information** – The system stores basic certificate details such as Certificate ID, student name, course, issuer, issue date, expiry date, and validity status.
- 8. Limited Dashboard Access** – The Institution Certificate Dashboard displays certificates associated with the connected institution wallet. It does not provide separate user accounts or role-based access for multiple staff members.
- 9. Expiry Status Depends on Blockchain Data** – Certificate expiry is determined using the expiry date stored in the smart contract. The institution must set or update this date correctly.
- 10. No External Verification Integration** – The prototype does not connect with external university systems, government databases, or accreditation authorities to independently validate certificate information.

These limitations indicate that BlockVerify is currently an academic prototype. Further development and testing would be required before using it as a production system for real institutions.

16. FUTURE SCOPE

BlockVerify can be improved further in the future to make the system more suitable for larger-scale use.

1. **Public Blockchain Deployment** – The system can be deployed on a public Ethereum-compatible network instead of only using Ganache.
2. **Better Institution Verification** – A system can be added to verify the real-world identity of the institution before assigning its blockchain wallet.
3. **More Certificate Details** – Additional certificate information can be stored according to the requirements of educational institutions.
4. **Larger-Scale Deployment** – The system can be extended to support multiple institutions and a larger number of certificates.
5. **Improved User Interface** – The interface can be further improved for easier use by institutions and certificate-verifying organizations.
6. **Mobile Support** – A mobile-friendly application can be developed to make certificate verification easier on smartphones.
7. **Integration with Institutional Systems** – In the future, BlockVerify can be connected with existing academic record systems to reduce manual data entry.

17. CONCLUSION

BlockVerify is a blockchain-based academic certificate verification system developed to make certificate management and verification simple, transparent, and reliable.

The system allows the authorized institution to issue certificates using a unique Certificate ID. Users can then enter the Certificate ID on the public verification page to check the certificate details, expiry date, and current validity status.

The institution can also update certificate information, revoke certificates that are no longer valid, and manage certificate expiry dates. The Institution Certificate Dashboard allows the institution to view issued certificates, check statistics, search and filter records, and view certificate details. These operations are controlled by the Solidity smart contract and restricted to the authorized institution wallet.

The project was developed using Solidity, Ganache, MetaMask, Ethers.js, HTML, CSS, and JavaScript. The prototype was tested for certificate issuing, verification, updating, revocation, duplicate certificates, invalid IDs, and unauthorized wallet operations. Additional dashboard and expiry-management features extend the certificate administration capabilities.

The completed prototype demonstrates the practical use of blockchain and smart contracts for academic certificate verification. It provides a simple foundation for managing certificate records and can be extended in the future for larger-scale use and integration with institutional systems.